ProEm Instruction Set

Draft v0.4

Paul Vincent Bezzina

26 September 2026

1 Summary

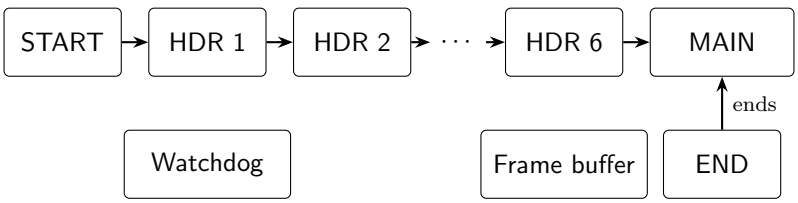
The RP2040 programs the emulator with a short list of byte instructions. The control unit decodes each instruction into one write of the frame chain's configuration registers (`cfg_*`); there is no instruction memory and nothing is replayed after reset. A program describes one frame: which block comes first, how many bits each block handles, which way each block talks, and what ends the frame. A `RUN` instruction then starts it.

Everything in this draft is implemented in the RTL and tested end to end at pin level: each test sends instruction bytes to the chip, drives or models the protocol's real lines, and reads the results back over the return UART. v0.4 adds the line codec behind `LINE` and `CLOCK`, the `OUT` and `LINK` instructions, auto-baud and the return link (Section 9). The changes from 25 September:

1. **Block index and direction in every block instruction.** Each instruction names the block it configures (`HDR 1`, `HDR 2`, ...) and carries its direction bit (0 = read, 1 = write). This is how the RTL already works internally; the ISA now makes it explicit.
2. **Run modes and block groups** NEW. Each block is tagged *always*, *read only* or *write only*. `RUN READ` runs the *always* and *read only* blocks; `RUN WRITE` runs the *always* and *write only* blocks. One program then covers both an I^2C register read and a register write.
3. **END pair compare** NEW. END can combine two pins instead of watching one. With D+ and D− this detects the USB end of packet (both low) and flags the illegal state (both high).
4. **Line decoding** NEW. `LINE` selects how a pin is decoded before the chain sees it: plain, NRZI (USB) or Manchester (10BASE-T), with optional bit unstuffing.
5. **Clock source** NEW. `CLOCK` selects where the chain's `tick` comes from: SYNC (an input clock pin), ASYNC (an internal bit timer, as in a UART), generated by the chip on an output pin, or recovered by the line decoder.

Items marked NEW were added on 25 September.

2 Frame chain recap

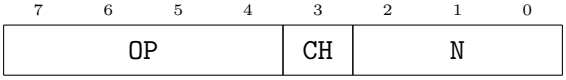


- Blocks run strictly in order, one bit per `tick`. A block that is not configured (or, with groups, not active in this run) is skipped.

- A *read* block samples its pin on each tick and compares or captures. A mismatch aborts the frame.
- A *write* block pushes its value into the frame buffer as a `wire` entry for the TX sender and succeeds in one clock.
- Read blocks only take ticks once every written bit ahead of them has left the chip (`line_free`), so a frame can write a request and read the reply.
- START and END have their own pins. The headers and MAIN share the data pin set by `MAIN`.
- Up to two chains (A and B) run side by side, for full-duplex SPI.

3 Instruction format

Every instruction starts with one byte: a 4-bit opcode and a 4-bit index. Operand bytes follow; multi-byte numbers are sent high byte first.



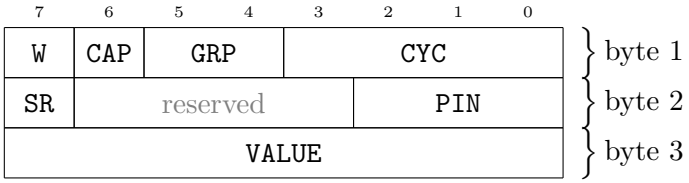
`CH` selects chain A (0) or chain B (1). `N` is the block number for `HDR` (1–6) or a small mode field for the other opcodes.

OP	Mnemonic	Operand bytes	Configures	RTL `cfg_sel`
0x0	NOP	0	–	–
0x1	START	3	START block	0
0x2	HDR n	3	Header block n = 1–6	1–6
0x3	MAIN	3	MAIN block and data pin	14
0x4	END	4	END block	15
0x5	WDOG	2	Watchdog limit	13
0x6	ORDER	0	Bit order, N[0]: 0 LSB, 1 MSB first	12
0x7	RUN	0	Start a frame, N[1:0] = mode NEW	–
0x8	HALT	0	Stop the chain and flush its buffer NEW	–
0x9	CLEAR	0	Unconfigure every block of the chain NEW	–
0xA	LINE	2	Line decoder for one pin NEW	–
0xB	CLOCK	4	Tick source NEW	–
0xC	DATA	1	Host entry, N = length − 1 NEW	–
0xD	OUT	1	Output drive, I^2C conditions, end of packet NEW	–
0xE	LINK	2	Return link divider (global) NEW	–
0xF	–	–	Reserved	–

4 Block instructions

4.1 START and HDR n

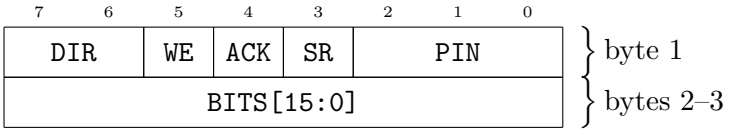
Both take the same three operand bytes.



Field	Meaning
W	Direction. 0 = read (compare against the line), 1 = write (send VALUE).
CAP	Read blocks only: also push the received value to the host.
GRP	NEW HDR only (START always runs). Group: 00 always, 01 read runs only, 10 write runs only, 11 reserved.
CYC	Field length in bits, 1–8. For START only, 0 means a *condition* START: succeed as soon as PIN equals VALUE[0], without using a tick (SPI chip select, the I²C START detector).
SR	NEW HDR only: a repeated START (Sr) just before this block. The chain sends the TX sender a wire entry of length 0, which means "Sr"; the block waits until it has gone out. Needed for I²C write-then-read.
PIN	START only: its own input pin. Headers ignore it and use the data pin set by MAIN.
VALUE	The value to send or compare, as a plain number in the bottom CYC bits. The control unit handles bit order and alignment.

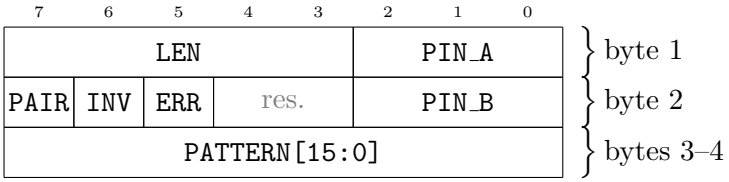
Headers must be configured in order from 1; the first unconfigured header ends the header list.

4.2 MAIN



Field	Meaning
DIR	00 read, 01 write, 10 NEW follow the run mode (read in RUN READ, write in RUN WRITE).
WE	Wait for END: finish when END matches instead of after BITS bits (read only).
ACK	Add one bit in the other direction after every byte. Reading: send 0, or 1 after the last byte. Writing: listen, and end the frame with host entry {0,02} (NACK) on a 1.
SR	NEW Repeated START before MAIN, as for HDR.
PIN	Data pin, used by MAIN and every header.
BITS	Data bits per frame, 1–65535.

4.3 END



END shifts one input bit into a 16-bit window on every end_tick and matches the last LEN bits (1–16) against PATTERN. It ends a MAIN running with WE, and after an abort it makes START wait until END has matched again. PAIR, INV, ERR and PIN_B are NEW.

Field	Meaning
PAIR = 0	Single pin: the window input is PIN_A. Used for 10BASE-T (raw line, pattern of all ones).
PAIR = 1	Pair compare: the window input is built from PIN_A and PIN_B (Section 6).
INV	Invert PIN_B before the compare.
ERR	Abort the frame and raise line_error when both inputs are 1 on an end_tick.

4.4 DATA

`DATA` puts one {`length, value`} entry into the chain's 8-entry host FIFO, which feeds a write MAIN. When the FIFO is full (`host_full`) further entries are dropped, so the RP2040 sends at most 8 ahead of `RUN` and paces the rest. The length (1–8) is `N` + 1 and the value is the one operand byte, so each entry costs two bytes on the serial link. Entries can be loaded before `RUN`.

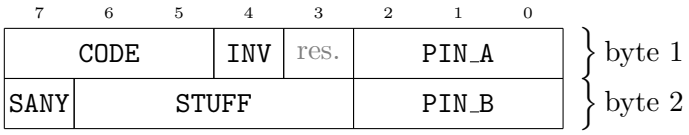
4.5 WDOG, ORDER

`WDOG` takes a 16-bit limit in clocks; the frame ends if no bit or host entry arrives for that long (0 = off). `ORDER` sets LSB or MSB first for every block of the chain and should come before `END`.

5 Line decoding and clock source

A chain needs two things besides its blocks: a bit on its data pin and a `tick` saying when that bit is valid. `LINE` sets how the bit is produced, `CLOCK` sets where the tick comes from. Both apply to the chain named by `CH`; `N` is unused.

5.1 LINE



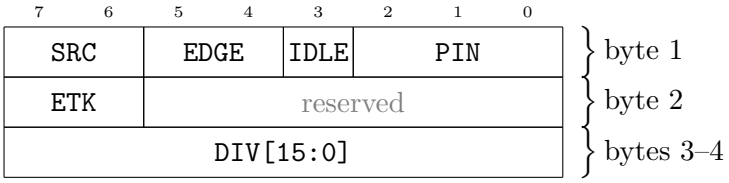
Every input pin passes through a two-flop synchroniser and one more register before any block sees it. With a timed clock source (ASYNC or RECOVER) the decoder works on pin `PIN_A` and its output *replaces* that pin for START, the headers and MAIN: a block whose `PIN` equals `PIN_A` reads decoded bits, one per `tick`. END and the other pins still see the raw lines, so END can watch the real end-of-frame state (the idle high of 10BASE-T, SE0 on USB). With SYNC or GENERATE the pins are used as they are.

`CODE`	Decoding	Used by
`000`	None: the pin is the bit	UART, SPI, I^{2}C, PS/2
`001`	NRZI: no change = 1, change = 0	Single-ended NRZI
`010`	NRZI on a differential pair: J/K from `PIN_A`, `PIN_B`; no bits while SE0 or SE1	USB (`PIN_A` = D+, `PIN_B` = D−)
`011`	Manchester, IEEE 802.3: low-to-high mid-bit = 1, high-to-low = 0	10BASE-T
`100`	Manchester, G. E. Thomas: the opposite convention	
`101`–`111`	Reserved	

- `INV` inverts the pin before decoding and the output after encoding (for USB full speed, where J is D+ high).
- `STUFF` = n removes the bit after n equal bits in a row, and gives it no `tick`, so the chain never sees it; 0 turns unstuffing off. USB: $n = 6$, counting 1s only (`SANY` = 0). CAN would use $n = 5$ counting runs of either value (`SANY` = 1). A stuffed bit with the wrong value aborts the frame with `line_error`.
- The encoder uses the same setting for what the chain sends: NRZ, NRZI (starting from J), or Manchester, with a stuff bit inserted after n equal bits. With code `010` it drives the pair: D+ on the data output, D− on the pair output.

- NRZ and NRZI sample in the middle of each bit and re-centre on every edge. Manchester decodes on the mid-bit edges: after each accepted edge it ignores edges for $\frac{3}{4}$ of a bit, so boundary edges between equal bits are skipped. This needs at least 5 clocks per bit: at 4, an edge can sit exactly between the two cases.

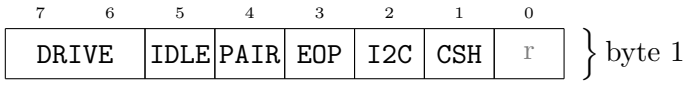
5.2 CLOCK



SRC	Mode	Where tick comes from	Fields used
00	SYNC	Edges of an input clock pin (SCK, SCL, PS/2 clock)	PIN = clock input, EDGE
01	ASYNC	The decoder's bit timer on LINE's PIN_A, sampling half a bit after each edge; transmit runs the same timer freely	DIV = clocks per bit
10	GENERATE	The chip makes the clock on the clock output; one tick per bit, on the edge EDGE names	PIN = clock read-back input (clock stretching with open drain), EDGE, IDLE, DIV = clocks per half period (≥ 4)
11	RECOVER	As ASYNC; the name used with coded lines (NRZI, Manchester)	DIV = nominal clocks per bit

- EDGE: 00 rising, 01 falling, 10 both. This covers the SPI modes: sample on one edge; the encoder changes the output on the other (SYNC) or at the start of each bit (GENERATE).
- IDLE: level of a generated clock between frames (SPI CPOL; 1 for I^{2}C SCL).
- A generated clock only runs while there is work: the TX sender has a bit waiting or a read block is waiting for one (listen). It never runs past the data.
- ETK selects end_tick: 00 same as tick, 01 every system clock, 10 the decoder's mid-bit strobe (every bit time, including stuffed bits).
- DIV is in system clocks (50 MHz). UART at 115 200 baud: 434. USB low speed: 33 (1.515 Mb/s, within ±1.5 %). 10BASE-T receive: 5. I^{2}C at 100 kHz generated: 250 (half period). Manchester transmit needs an even DIV, so 10BASE-T transmit needs a 40 or 60 MHz clock.
- In I^{2}C mode (OUT) a generated SCL is low for the first half of each bit and high for the second, whatever IDLE says; the bus idles high between transactions.

5.3 OUT



Each chain has a data output, a pair output, a clock output (each with its own output enable) and a select output.

Field	Meaning
`DRIVE`	`00` off (after reset and `CLEAR`), `01` push-pull while a packet is being sent, `10` open drain on data and clock (drive low for 0, release for 1), `11` push-pull always.
`IDLE`	Data level between frames with `DRIVE = 11` (1 for a UART line).
`PAIR`	Drive the complement on the pair output (USB D−).
`EOP`	At the end of each frame send an end of packet: two bit times of SE0 then one of J on a pair (USB), or two bit times high (10BASE-T).
`I2C`	With a generated clock: send START before the first byte, Sr for a repeated-START entry, and STOP after each frame.
`CSH`	Select output active high (0: active low, as SPI chip select). It is active from the frame's START until its last bit has gone out.

The end of a frame is marked by the frame's status entry (below), which passes through the frame buffer in order behind the frame's last bits. STOP and the end of packet therefore always come after the frame's own bits and before the next frame's.

5.4 `LINK` and the return link

`LINK` (`0xE`, global, 2 operand bytes) sets the return UART's clocks per bit; 0 uses the rate measured by auto-baud. At 50 MHz, 4 clocks per bit is 12.5 Mbaud, which the RP2040's PIO can receive.

Auto-baud. After reset the first byte on the instruction line must be `0x55`. The chip times its first five falling edges (8 bit times), sets its receive rate from that, and does not pass the byte on.

Return format. Every message is two bytes, sent with 1 start and 1 stop bit:

First byte	Second	Meaning
`10 C 0 LLLL`	value	Host entry from chain `C`, length `L` (0–8; length 0 value `02` = NACK)
`10 C 0 1111`	flags	End of a frame on chain `C`: bit 0 complete, 1 abort, 2 timeout, 3 line error, 4 overflow
`11 000000`	flags	Link events: bit 0 `rx_error`, 1 `cmd_error`, 2 `DATA` dropped on A, 3 on B

A frame whose host entries arrive faster than the return link can carry them fills the 16-entry frame buffer; further entries are dropped and the frame's status has the overflow bit set. Written bits are never dropped: write blocks wait for space.

6 END pair compare

With $a =$ `pins[PIN_A]` and $b' =$ `pins[PIN_B]` $\oplus$ `INV`:

$$\begin{aligned} \text{both_high} &= a \wedge b' \\ \text{equal} &= \neg(a \oplus b') \\ \text{window input} &= \text{equal} \wedge \neg\,\text{both_high} \;=\; \neg a \wedge \neg b' \\ \text{line_error} &= \texttt{ERR} \wedge \text{both_high} \quad \text{(checked on } \texttt{end_tick}\text{)} \end{aligned}$$

For USB low speed, $a =$ D+, $b =$ D−, `INV` $= 0$:

D+	D−	AND	XNOR	Window input	Line state
0	1	0	0	0	J (idle, data)
1	0	0	0	0	K (data)
0	0	0	1	**1**	SE0: end of packet
1	1	1	1	0	SE1: illegal, `line_error`

- `end_tick` must be the decoder's mid-bit sample strobe, not every clock. D+ and D− do not switch at exactly the same instant, so for a few nanoseconds around each J/K change both can read low (or high). Sampling mid-bit ignores that.
- `LEN` = 2, `PATTERN` = `0x0003`: END matches after two bit times of SE0, which is exactly what the sender drives.
- `end_tick` runs on every bit time, including stuffed bits, because it comes from the bit timer rather than from the unstuffed data `tick`.
- With `INV` = 1 the window input is 1 when `PIN_A` is low and `PIN_B` is high (for USB low speed with D+, D−: the J idle state), so END can wait for the line to return to idle; `ERR` should then be off.
- The pair compare looks at levels, so it cannot see the I^2C STOP condition, which is an edge (SDA rising while SCL is high). STOP and START detection stay in the pin block.

This replaces the separate `se0` pin proposed earlier: END reads the raw D+ and D− directly, while `LINE` gives the blocks the decoded bits.

7 Run modes and block groups

`RUN` starts a frame on the chain named by `CH`. `N[1:0]` selects the mode:

`N[1:0]`	Mode	Blocks that take part
`00`	`RUN`	`GRP = 00` only
`01`	`RUN READ`	`GRP = 00` and `01`; MAIN with `DIR` = follow reads
`10`	`RUN WRITE`	`GRP = 00` and `10`; MAIN with `DIR` = follow writes

- A chain whose START writes starts one frame straight away (this replaces the `go` input). In plain `RUN` mode it then also starts a new frame whenever host data is waiting, which is what a UART TX stream needs. In `RUN READ` and `RUN WRITE` it only starts on `RUN`, so `DATA` for the next transaction can be loaded early.
- A chain whose START reads is armed and waits for its pattern or condition; it stays armed after each frame.
- The mode stays set until the next `RUN`, so a listening chain keeps re-arming in the same mode. `RUN` is meant for between frames.
- `HALT` stops the frame in progress, drops anything not yet released from the frame buffer, empties the host FIFO and disarms the chain. `CLEAR` resets the chain completely: every block, both buffers, the bit order and its `LINE` and `CLOCK` settings.
- The router already skips unconfigured blocks. Groups add one more condition to that skip: the block's group must be active in this run. The per-block `W` bit is unchanged.

Why groups rather than "activate every READ block". A single I²C transaction mixes both directions: the chip writes the address, reads the ACK, writes the register, reads the ACK, and so on. Every block's direction is fixed by the protocol, and the blocks must still run in order. So the direction must stay per block (the `W` bit). What changes between a register read and a register write is *which* blocks run, and which way MAIN goes. Groups and the follow mode for MAIN express exactly that, so the RP2040 loads one program and chooses the transaction with `RUN READ` or `RUN WRITE`.

8 Example programs

8.1 I²C register read and write, one program

Device 0x48, register 0x05, chip as controller, chain A. SDA and SCL are open-drain pads: the chip drives them through the data and clock outputs and reads them back on pins 0 and 1.

Instruction	Bytes	W	Group	Job
`ORDER MSB`	`61`			MSB first
`LINE`	`A0 00 00`			No decoding
`CLOCK`	`B0 89 00 00 FA`			Generate SCL, 100 kHz, read back on pin 1
`OUT`	`D0 A4`			Open drain, I²C conditions
`WDOG 4000`	`50 0F A0`			
`START`	`10 88 00 90`	1	always	Address 0x48 + W
`HDR 1`	`21 01 00 00`	0	always	ACK
`HDR 2`	`22 88 00 05`	1	always	Register 0x05
`HDR 3`	`23 01 00 00`	0	always	ACK
`HDR 4`	`24 98 80 91`	1	read	Sr, address 0x48 + R
`HDR 5`	`25 11 00 00`	0	read	ACK
`MAIN`	`30 90 00 10`	follow		16 bits, `ACK` on

- `RUN WRITE` (`72`) with two data bytes in the host FIFO: `S 90 A 05 A d0 A d1 A P`.
- `RUN READ` (`71`): `S 90 A 05 A Sr 91 A` then two bytes read, ACK then NACK, `P`.

8.2 USB low-speed receive

Chain A, LSB first. Pins: 0 = D+, 1 = D−. `LINE` decodes the pair onto pin 0 for the blocks; END reads the raw pins.

Instruction	Bytes	Job
`ORDER LSB`	`60`	
`LINE`	`A0 40 31`	NRZI pair, D+ = 0, D− = 1, unstuff after six 1s
`CLOCK`	`B0 C0 80 00 21`	Recover, 33 clocks per bit, `end_tick` = mid-bit strobe
`START`	`10 08 00 80`	Read 8 bits: SYNC `00000001`
`MAIN`	`30 20 FF FF`	Read on pin 0 until END
`END`	`40 10 A1 00 03`	Pair D+/D−, `ERR` on, 2 bits of SE0
`WDOG 200`	`50 00 C8`	
`RUN READ`	`71`	Arm

MAIN reads the PID as its first byte: a read header always compares, so it can only accept one PID value, not capture any. The host checks the PID complement and the CRC5 or CRC16, as it checks the CRC for Ethernet.

8.3 USB low-speed transmit

Chain B sends SYNC, a DATA1 PID and data from the host FIFO, with bit stuffing and an end of packet:

Instruction	Bytes	Job
`LINE`	`A8 40 31`	NRZI pair, stuff after six 1s
`CLOCK`	`B8 C0 00 00 21`	33 clocks per bit
`OUT`	`D8 58`	Drive while sending, pair, end of packet
`START`	`18 88 00 80`	Write SYNC
`HDR 1`	`29 88 00 4B`	Write PID
`MAIN`	`38 40 00 20`	Write 32 bits from `DATA`
`DATA` × 4, `RUN WRITE`	`CF xx ... 7A`	Load the data, then send

8.4 Other standards

Standard	Program
UART RX	`LINE` none on RX; `CLOCK` ASYNC, `DIV` = 434; `START` read 1 bit = 0; `MAIN` read 9 bits. Host checks the stop bit.
UART TX	`CLOCK` ASYNC, `DIV` = 434; `OUT` push-pull always, idle 1; `START` write 1 bit = 0; `MAIN` write 9 bits; `DATA` {8, byte}, {1, 1}.
SPI controller	Chain A: `CLOCK` GENERATE; `OUT` push-pull; `START` write with `CYC` = 0; `MAIN` write on MOSI; chip select on the select output. Chain B: `CLOCK` SYNC on SCK read back, condition `START` on CS read back, `MAIN` read on MISO.
SPI target	Chain B reads MOSI with `CLOCK` SYNC on the master's SCK; chain A answers on MISO with `CLOCK` SYNC too, changing on the other edge.
PS/2 device to host	`CLOCK` SYNC, falling edge of the device clock; `START` read 1 bit = 0; `MAIN` read 10 bits.
10BASE-T RX	`LINE` Manchester IEEE; `CLOCK` RECOVER, `DIV` = 5, `end_tick` every clock; `START` read 8 bits `0xD5`; `HDR 1–6` compare the destination MAC; `MAIN` read with `WE`; `END` on the raw line, 8 ones; `LINK` 4.
Manchester TX	`LINE` Manchester; `CLOCK` RECOVER, even `DIV`; `OUT` drive while sending, `EOP`; preamble from `START` and six write headers, then `DATA`.

9 Changes in v0.4

- Line codec per chain behind `LINE` and `CLOCK`: NRZ, NRZI, NRZI pair, Manchester, bit stuffing both ways, all four clock sources.
- New `OUT` (`0xD`): drive mode, idle level, pair output, end of packet, I^2C START/Sr/STOP, select polarity. Outputs are off after reset.
- New `LINK` (`0xE`) and a return UART carrying host entries, a status per frame and link events.
- Auto-baud on the instruction line; the first byte after reset is `0x55`.
- In GENERATE mode `PIN` is the clock read-back input used for clock stretching.
- Write blocks wait when the frame buffer is almost full; received entries that do not fit are dropped and reported.

10 Changes in v0.3

- New `DATA` instruction (`0xC`) carries write data over the same serial link; the pin block moves to `0xD`–`0xF`.
- `GRP` and `SR` apply to headers and MAIN only; START ignores them.
- Repeated START is a `wire` entry of length 0 to the TX sender. A write START with `CYC` = 0 (used for SPI) sends nothing.
- `RUN READ` and `RUN WRITE` start a write-START chain only on `RUN`; plain `RUN` also starts frames when host data waits.
- Corrected the meaning of END with `INV` = 1, and dropped the capturing header from the USB example.
- `cmd_error` pulses for a reserved opcode or `HDR 0`/`HDR 7`; `rx_error` pulses when a byte's stop bit is low.

11 Status and open items

Feature	State	Note
Frame chain, control unit, groups, `RUN` modes, END pair, `SR`, `HALT`, `CLEAR`, `DATA`	In RTL, tested	1,800 random programs at pin level plus the tests below
Line codec: NRZ, NRZI pair, Manchester, stuffing, SYNC, ASYNC, GENERATE, RECOVER	In RTL, tested	UART, SPI, I^2C, PS/2, 10BASE-T RX, Manchester TX, USB low speed RX and TX
`OUT`, `LINK`, auto-baud, return link	In RTL, tested	Every test reads its results back over the return UART
NRZI single-ended, Manchester Thomas, CAN-style stuffing	In RTL, not tested	Same paths as the tested codes

Open questions:

1. I^2C *target* mode: the direction after the address depends on the R/W bit the other side sends, and the chain cannot branch on a bit it has just read. A START/STOP detector on the input side would also be needed.
2. Header capture of any value (a "don't care" read header), for fields such as the USB PID.
3. The return link carries two bytes per host entry. Full-size 10BASE-T frames (64 bytes and up) arrive faster than even a 4-clock link can return them; they are reported as overflows.
4. The host FIFO holds 8 entries and the instruction link is slow, so a transmitted packet is limited to what the write blocks and 8 `DATA` entries hold (about 14 bytes).
5. A host entry waiting for the return link holds up the wire entries behind it; I^2C just stretches SCL, but a timed transmit would pause.